C06｜结果对了以后，怎样判断慢在哪里

本章的线索是：**固定正确答案 → 看数据库的访问计划 → 在相同条件下重复计时 → 判断收益是否值得维护成本**。计划和耗时回答不同问题，都不能替查询判断客户、行与顺序是否正确。

怎样先固定这次要比较的工作?

当前问题	关系或短用法	记到什么程度
日志报一次慢，就能确定数据库有问题吗?	先找同类 operation 的重复现象，按 request ID 对上处理；总计时可能含连接和结果整理。	先认范围与稳定性，不把一个样本当结论。取有代表性的数据、输入与 SQL 复现。
“最近 20 条”为什么只验行数不够?	`WHERE customer_id=? ORDER BY starts_at DESC, id DESC LIMIT 20`，同时约定客户、先后与并列次序。	**比较的是正确完整行及顺序。** 20 行仍可能属于错误客户或拿错边界。
错误 JOIN 很快，能开始优化吗?	`ON c.id=c.id` 没约束预约归属，两客户与三预约变六行；先改 `ON a.customer_id=c.id`，再建立正确查询的基线。	先修结果含义；DISTINCT 和索引都不能为错误对应作担保，错误 SQL 的旧耗时也不是正确基线。

在本章独立四列表中，目标客户有 30 条；约定答案是 `100030` 到 `100011`，同日的 `100030` 在 `100029` 前，状态不排除任何一条。这个固定结果用来比较前后，样例编号不需要记。

回看：「“最近 20 条”究竟应该是哪 20 条？」「如果结果数量本来就错了，应该先改哪里？」。

索引改变了哪一段工作，计划能说明到哪里？

当前问题	关系或短用法	记到什么程度
怎样看数据库准备怎样查？	在同一 SELECT 前加 `EXPLAIN QUERY PLAN`；保持查询与参数一致，读表名、索引名和使用条件。	逐渐认熟计划入口；它描述访问路线，不是实际耗时，也不是业务正确性判定。
SCAN 是错误，SEARCH 就一定快吗？	SCAN 表示遍历路线，也可能沿索引；SEARCH 表示按条件定位部分行。	**结合具体对象与条件解释**，不把英文词当成绩；输出格式随 SQLite 版本可能变化。
按客户建索引，为何排序仍在？	本例单列索引帮助定位目标客户的 30 条；它没有保证完成 `starts_at DESC, id DESC`，所以仍可能有 `USE TEMP B-TREE FOR ORDER BY`。	看懂“少访问无关行”与“减少排序”是不同收益。出现临时排序不等于客户索引没作用。
索引是另一份业务表吗？	`CREATE INDEX idx_appointments_customer_id ON appointments(customer_id)` 建立随原表维护的查找结构；应用仍发同一 SELECT。	认熟“为哪张表、哪列提供路线”；不是新增预约，也不需把 SELECT 改查索引名。

完整前后计划与重复计时代码查原章 `c06_history.py`。索引只是具体候选；是否采用及有多少收益，由当前计划和实测回答。

回看：「表里另外有十万条预约时，它怎样找到这位客户？」「计划变了，怎样知道这次改变有没有用？」。

数字更小，怎样判断是否值得保留？

当前问题	关系或短用法	记到什么程度
两边计时怎样才可比较？	相同 SQL、参数、数据、取行范围、预热和重复次数；每轮确认完整结果相同，保留原始样本。	先守同一工作与口径，再比较数字；不能一边只计 execute，一边计到取完行。
中位数能排除所有波动吗？	median 减少极端值对代表值的影响，但不表示最慢请求，也不消除缓存和后台负载影响。	认熟代表值的用途和边界；差异与波动相当时，结论是当前收益不明确。
小表、少数不同值，就一定不需要索引吗？	看当前条件实际匹配多少行，也就是缩小范围的能力；少数状态中某一个很稀少，仍可能有收益。	**选择性联系当前输入与分布。** 不只数不同值，也不背“每列都建 / 某类永远不建”。
读得更快，代价在哪里？	索引占空间；新增、删除行或修改索引相关列需维护条目。结合读取频率与写入负担判断。	有收益才考虑保留；本章内存读取实验没量化磁盘空间、写入成本或整次 HTTP 等待。
换回 C03、C04，也照抄这个索引吗？	那里的预约经 tickets 找客户，没有本章直接的 `appointments.customer_id`；需拿实际 JOIN、过滤、排序重新解释和测量。	**迁移的是判断方法，候选必须服从真实结构。** 同名业务问题不保证同一 SQL。

本章计时包含执行查询并 `fetchall()` 取完结果，不包含建库、生成数据、创建索引、断言和 JSON 输出；不能与 C05 含连接使用的事件耗时直接相减归因。它使用源码说明的 sqlite3 旧式事务控制写法，也不是 C04 显式 SQL 事务的替代模板。

多列、覆盖、部分索引，在单列方案后仍有明确瓶颈时再查；两连接争用要回到事务、锁与受控复现；空间回收才考虑 VACUUM。这些都不是遇到慢查询就执行的一套命令。

回看：「计划变了，怎样知道这次改变有没有用？」「索引读得快一些，就应该留下吗？」「回到完整应用时，哪些结论可以带走？」。

读到这里可以把 C 册重新接成一个判断：**事实由哪里保存，结果一行代表什么，成功必须共同留下哪些状态，失败后能凭什么证据继续。** 全套配套的 schema 与接口以它自己的文件为准；需要整体验证时，再回看「运行之前，先知道会改变什么」与「怎样知道不是只把测试运行了一遍？」。

下一册遇到外部供应商时，本地 ROLLBACK 不能自动撤回对方已经执行的动作。数据库的提交边界明确以后，才有依据继续判断外部失败、等待和重复请求。